CS-338 Intro to the Theory of Computation

- Lecture #16
- **Conclusion**

- Prof Pietro S. Oliveto
  Department of Computer Science and Engineering
  Southern University of Science and Technology (SUSTech)
- olivetop@sustech.edu.cn
  https://faculty.sustech.edu.cn/olivetop
- Office: Room 113

Overview

Last time:

- Cook-Levin Theorem: SAT is NP-complete
- $3SAT$ is NP-complete

Today:

- Space complexity
- NP-hardness
- How to cope with intractability
- Revision and Mock exam

Time Complexity: Quick Review

Defn: B is NP-complete if

1) $B \in$ NP
2) For all $A \in$ NP, $A \leq_P B$

If B is NP-complete and $B \in$ P then P = NP.

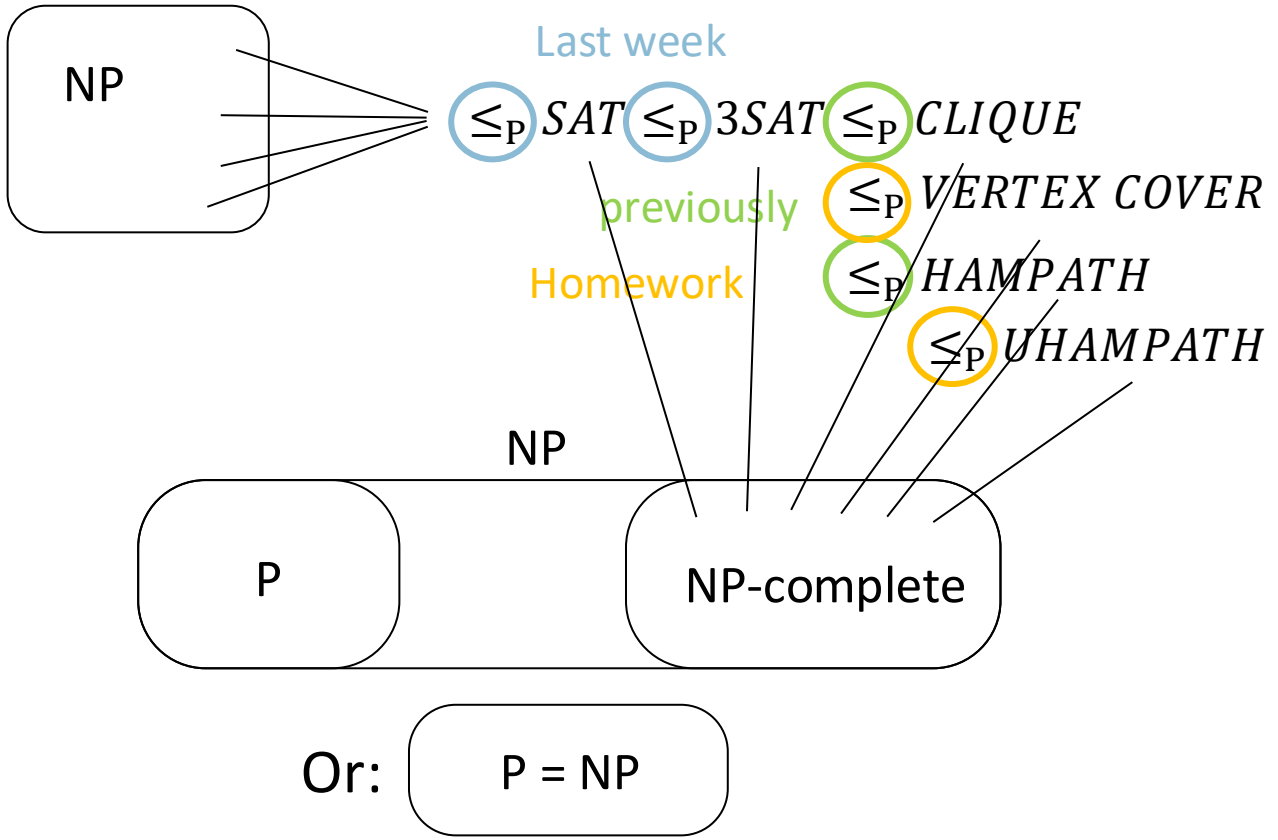
Importance of NP-completeness

1) Evidence of computational intractability.
2) Gives a good candidate for proving P ≠ NP.

To show some language C is NP-complete, show $3SAT \leq_P C$.

or some other previously shown NP-complete language

What about space?



SPACE Complexity Classes

Defn: Let $f: \mathbb{N} \to \mathbb{N}$. Say TM M runs in space $f(n)$ if M always halts using maximum $f(n)$ tape cells on all inputs of length n.

Typically space complexity uses asymptotic notation. Let $f: \mathcal{N} \to \mathcal{R}^+$

Defn: $\mathrm{SPACE}(f(n)) = \{B \mid B$ is decided by some $O(f(n))$ space deterministic TM $M\}$

Defn: $\mathrm{NSPACE}(f(n)) = \{B \mid B$ is decided by some $O(f(n))$ space nondeterministic TM $M\}$

Theorem

SAT is in $SPACE(O(n))$

Proof

Just evaluate on the same portion of the tape each of the 2^n truth assignments.

Space Complexity

Defn: $PSPACE = \bigcup_k SPACE(n^k)$

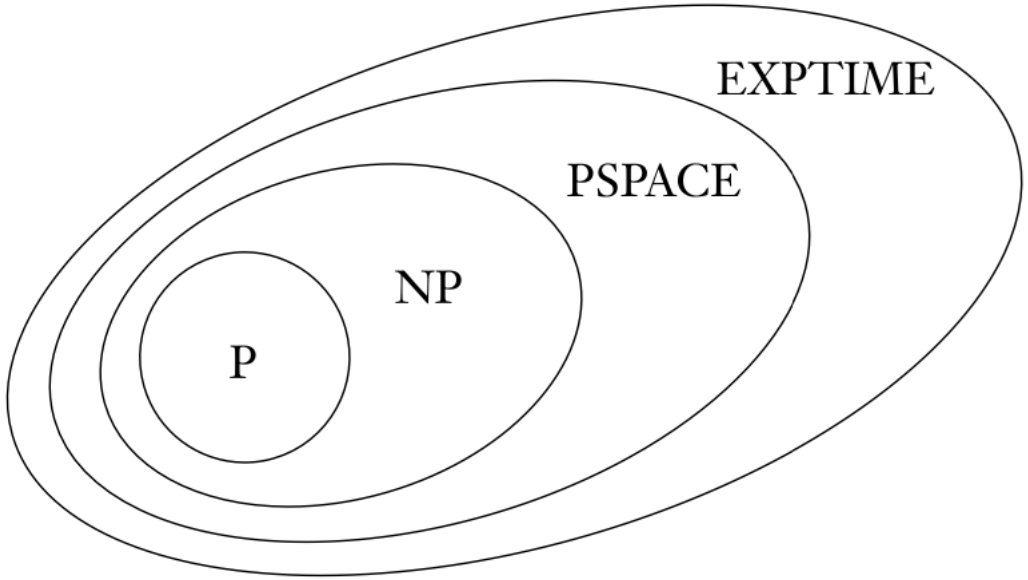
Defn: $NPSPACE = \bigcup_k NSPACE(n^k)$

Savitch's Theorem: For any function $f: \mathcal{N} \to \mathcal{R}^+$, where $f(n) \geq \log n$,

$$NSPACE(f(n)) \subseteq SPACE(f^2(n))$$

Corollary:

$PSPACE = NPSPACE$



$P \subseteq PSPACE$

$NP \subseteq NPSPACE$

=> $NP \subseteq PSPACE$

$P \neq EXPTIME = \bigcup_k 2^{n^k}$ (known)

(therefore at least one of the three containments is proper (we don't know which!))

NP-hardness

Defn: B is NP-complete if
1) $B \in$ NP
2) For all $A \in$ NP, $A \leq_P B$

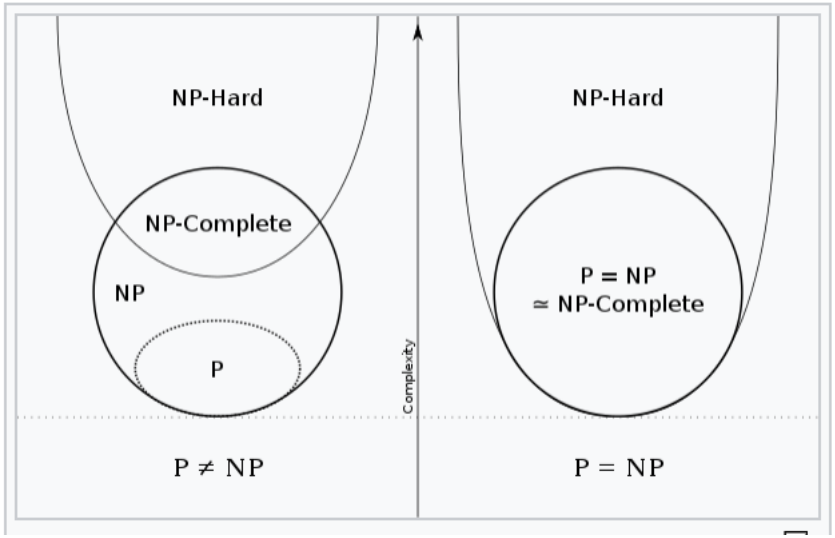
NP problems are decision problems: $w \in A, w \notin A$

SAT: "is ϕ satisfiable?"
VERTEX COVER: "Does G have a cover of size k?"

Defn: A problem B is **NP-hard** if
1) For all $A \in$ NP, $A \leq_P B$

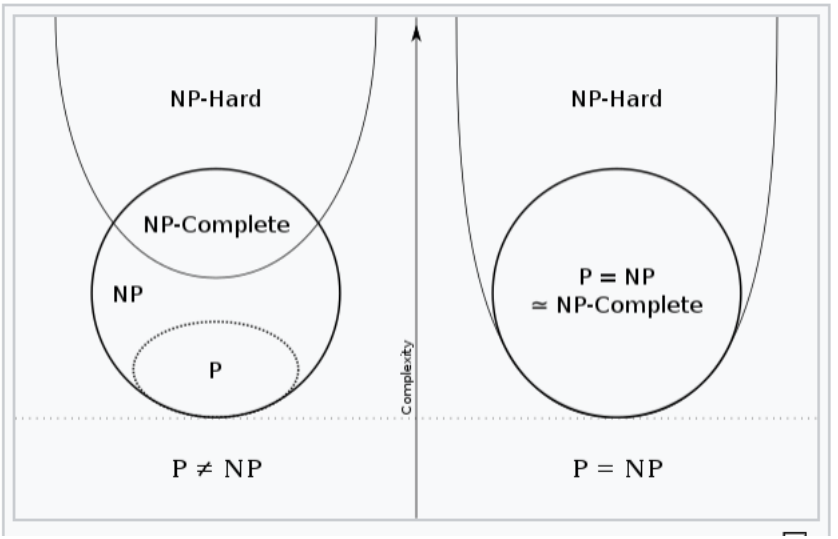
- NP-hard problems do not have to be in NP ("at least as difficult as problems in NP")
- Still finding a poly-time algorithm for an NP-hard problem => P=NP

Defn: B is NP-complete if
1) $B \in$ NP
2) B is NP-hard



Also search and optimization problems are included —>

Decision vs Optimisation problems



Defn: A problem B is **NP-hard** if

1) For all $A \in$ NP, $A \leq_{\mathrm{P}} B$

Decision problem

SAT: "is ϕ satisfiable?"

VERTEX COVER: "Does G have a cover of size k?"

Optimisation problem

We seek for the best solution among all possible solutions.

MAXSAT: "find the maximum number of satisfiable clauses in ϕ"

$MAXSAT = \{\phi, k \mid \phi$ is a Boolean formula with *at least* k satisfiable clauses$\}$

MIN-VERTEX COVER: "find the cover of G minimum size"

$MIN - VERTEX - COVER = \{G, k \mid G$ is an undirected graph with a cover of size at most $k\}$

Many interesting problems are not solvable in polynomial time unless P=NP. What do we do?

Approximation algorithms

Many interesting problems are not solvable in polynomial time unless P=NP. What do we do?

Relax the problem: do we really need an optimal solution ?

Is a nearly optimal solution good enough? -> Approximation algorithms

Example: Algorithm A for Vertex Cover

"On input $\langle G \rangle$, where G is an undirected graph:

1. Repeat the following until all edges in G touch a marked edge:
2. Find an edge in G untouched by any marked edge.
3. Mark that edge.
4. Output all nodes that are endpoints of marked edges."

Theorem: Algorithm A is an $O(m)$ 2-approximation algorithm for Vertex Cover .

Proof: Let opt^* be the optimal solution and X be the one produced by A:

1) X is a cover (every edge is covered)

2) for each edge X contains at most two nodes (opt^* has at least one per edge)

Randomised algorithms

Many interesting problems are not solvable in polynomial time unless P=NP. What do we do?

Relax the problem: Instead of coming up with a deterministic procedure that runs in poly time, just make random decisions

Example: Algorithm B for *MAX-3SAT*

1. Assign $x_i = 1$ with probability 1/2 for each variable

Theorem: Algorithm B is an $O(n)$ at least 7/8-approximation algorithm in expectation for *MAX-3SAT*.

Proof:

1) The probability each clause is not satisfied is $(\frac{1}{2})^3 = \frac{1}{8}$, thus satisfied with probability $\frac{7}{8}$

2) Let X_i be an indicator variable saying whether clause C_i is satisfied. Then

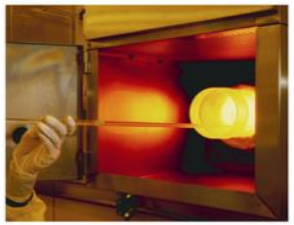
3) $E[X_i] = 0 \cdot \frac{1}{8} + 1 \cdot \frac{7}{8} = \frac{7}{8}$, and

4) $E[X] = E[\sum_{i=1}^{n} X_i] = \sum_{i=1}^{n} E[X_i] = \frac{7}{8} n$

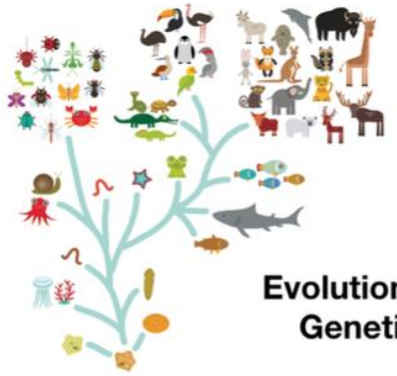
Bio-Inspired Search Heuristics

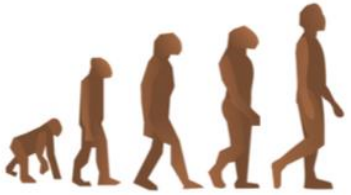
Many interesting problems are not solvable in polynomial time unless P=NP. What do we do?

Relax the problem: Instead of coming up with a deterministic procedure that runs in poly time, just apply a **general purpose algorithm**



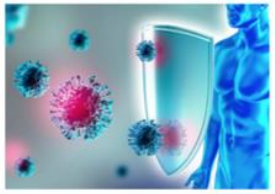
Simulated Annealing





Evolutionary Algorithms
Genetic Algorithms

Ant Colony Optimisation



Artificial immune Systems

Swarm Optimisation

- Can be applied to virtually any optimization problem
- Do not require deep understanding of the problem
- Often are inspired by optimization processes that have shown to be successful in nature

Quick review of today

1. $SPACE(T(n)) \quad - PSPACE - NPSPACE$
2. NP-hardness
3. Alternative options:
    a. Approximation algorithms
    b. Randomised algorithms
    c. General purpose search heuristics